Formalização de Limites Inferiores na Distribuição de Primos via Complexidade de Kolmogorov

Códigos Autodelimitados e o Método da Incompressibilidade em Lean 4

Gabriel Resende Machado

INF2290: Tópicos em Teoria da Computação IV – Formalização em Lean

Departamento de Informática, PUC-Rio

12 de Novembro de 2026

Sumário

1. Introdução
2. Problema de Pesquisa
3. Objetivos Geral e Específicos
4. Resultados e Entregáveis
5. Cronograma
6. Referências

Sumário

1. Introdução
2. Problema de Pesquisa
3. Objetivos Geral e Específicos
4. Resultados e Entregáveis
5. Cronograma
6. Referências

O Desafio dos Limites Inferiores

Na Teoria da Computação, há uma assimetria fundamental entre dois tipos de problemas:

- **Limites Superiores:** para estabelecer um limite superior, basta demonstrar um algoritmo, um circuito ou uma codificação que resolva o problema em espaço ou tempo $T \in O(f(n))$;
  - O algoritmo *MergeSort* ordena n itens com $O(n \log n)$ comparações (afirmação existencial $\exists$).
- **A Dificuldade Assintótica dos Limites Inferiores:** estabelecer um limite inferior $T \in \Omega(f(n))$ exige uma prova universal negativa, *i.e.* demonstrar que **nenhum** algoritmo possível, sob um determinado modelo computacional, pode ser mais rápido ou compacto que um dado limiar.
  - Exibir um caminho de 10 km prova que a distância é no máximo 10 km; provar que ela é no mínimo 10 km exige descartar todos os caminhos possíveis (afirmação universal $\forall$).

O Desafio dos Limites Inferiores

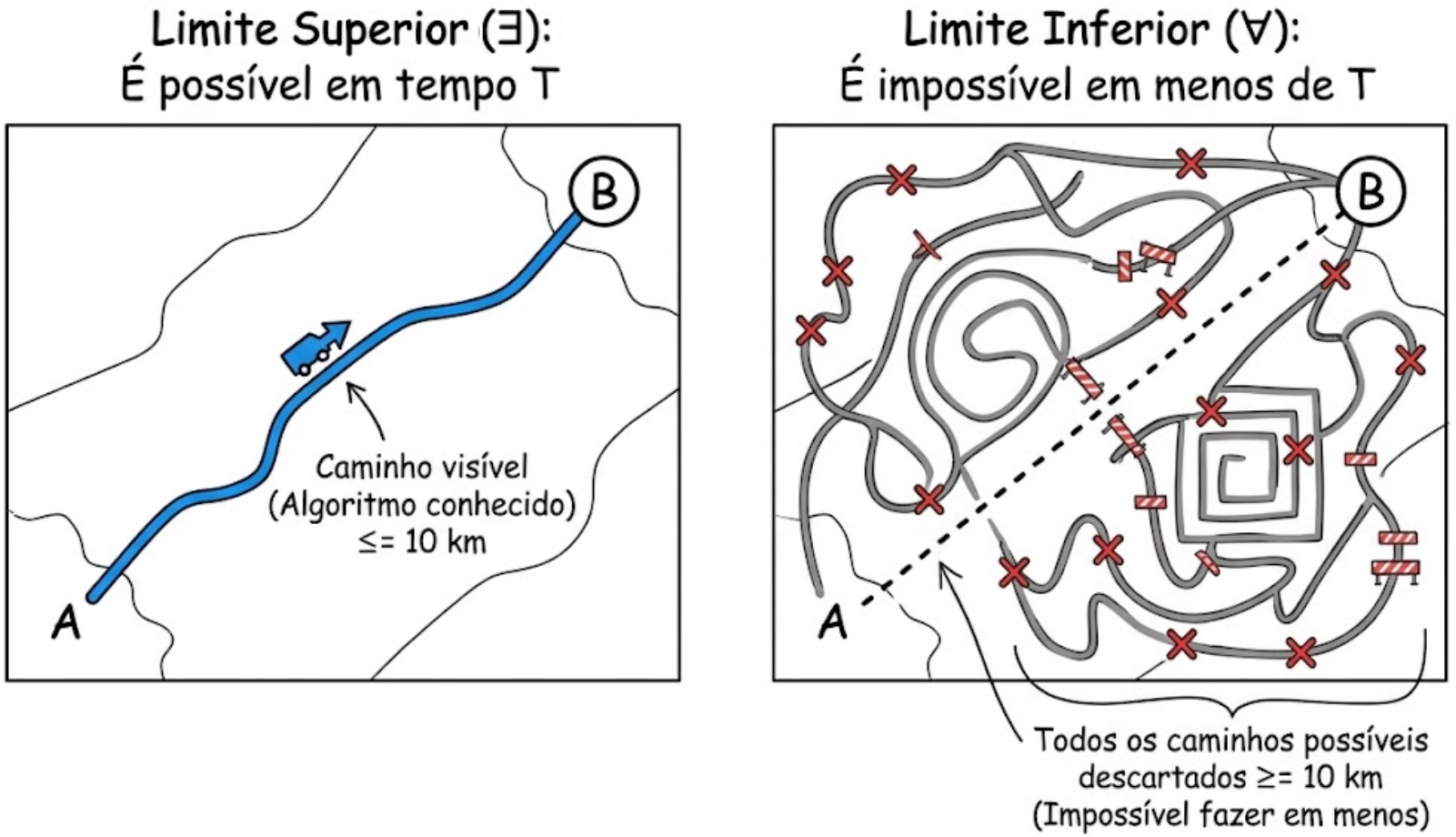


Figura 1: A assimetria na Teoria da Complexidade. À esquerda, um limite superior prova-se encontrando um único caminho válido. À direita, um limite inferior exige demonstrar que nenhum caminho no universo de possibilidades consegue quebrar a barreira estabelecida.

Regularidade vs. Aleatoriedade

- Um objeto é aleatório se não for possível gerar sua sequência por meio de um programa menor do que ela mesma;
- **Objetos aleatórios são comumente chamados de imcompressíveis via Complexidade de Kolmogorov.**

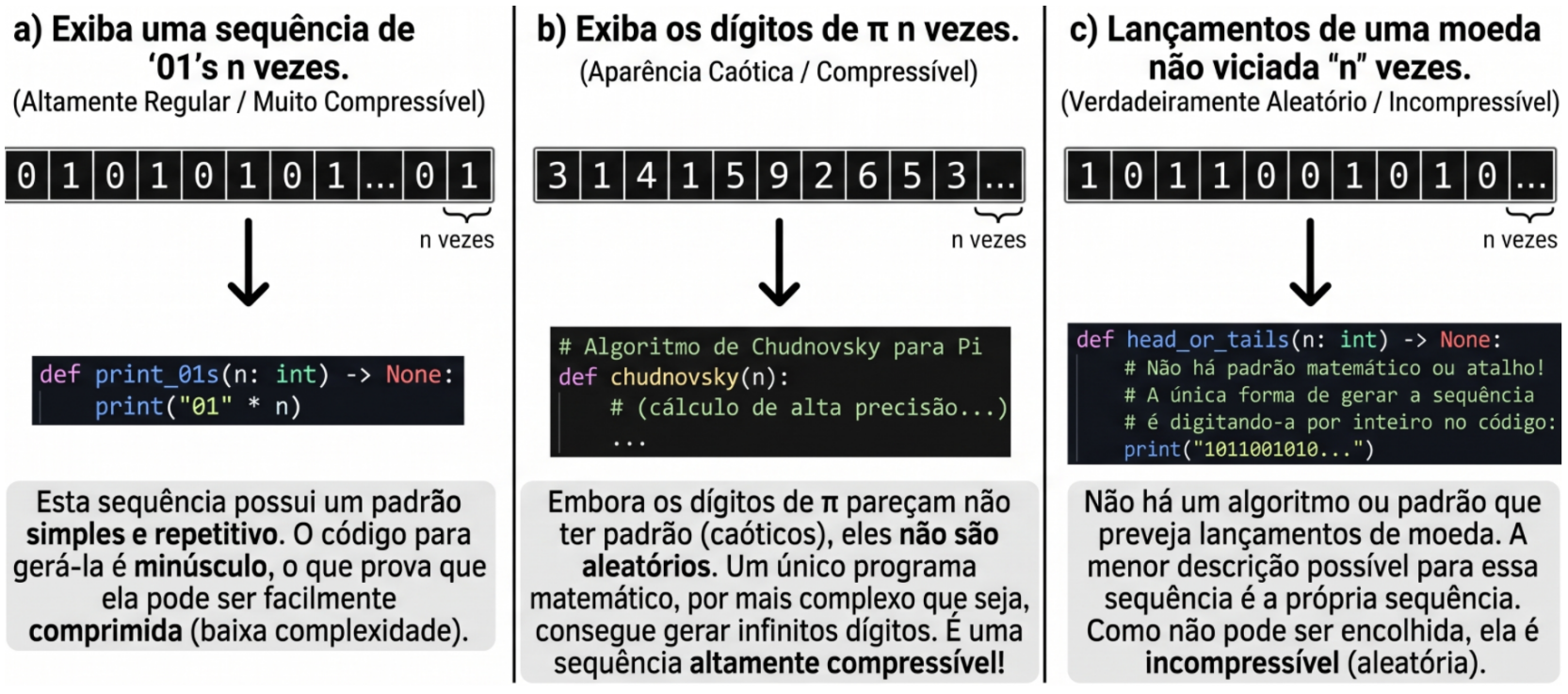


Figura 2: Regularidade vs. Aleatoriedade na Complexidade de Kolmogorov: a incompressibilidade algorítmica como a verdadeira medida do que é aleatório.

A Complexidade de Kolmogorov

A Complexidade de Kolmogorov $K(x)$ é o tamanho (em bits) do menor programa que gera x, tal que $|x| = n$.

Definição Formal

Na Figura 2, nos casos *a*) e *b*), as strings de n bits obedeciam a algum padrão ou comportamento reproduzível por meio de um algoritmo de custo adicional c produzir uma saída menor do que n. Daí, tem-se:

$$K(x) < n + c \tag{1}$$

Contudo, no caso *c*), não há como reproduzir a saída de n lançamentos de moeda pela sua intrínseca natureza não determinística, a não ser por um algoritmo específico que exiba a própria saída n de lançamentos. Portanto:

$$K(x) \geq n - c \tag{2}$$

Como aqui o objetivo é caracterizar sequências que não podem ser significativamente comprimidas, c representa a quantidade máxima de bits que admite-se perder em relação ao comprimento original da sequência.

O Princípio da Casa dos Pombos

Objetos imcompressíveis – ou Kolmogorov-Aleatórios – violam o Princípio da Casa dos Pombos.

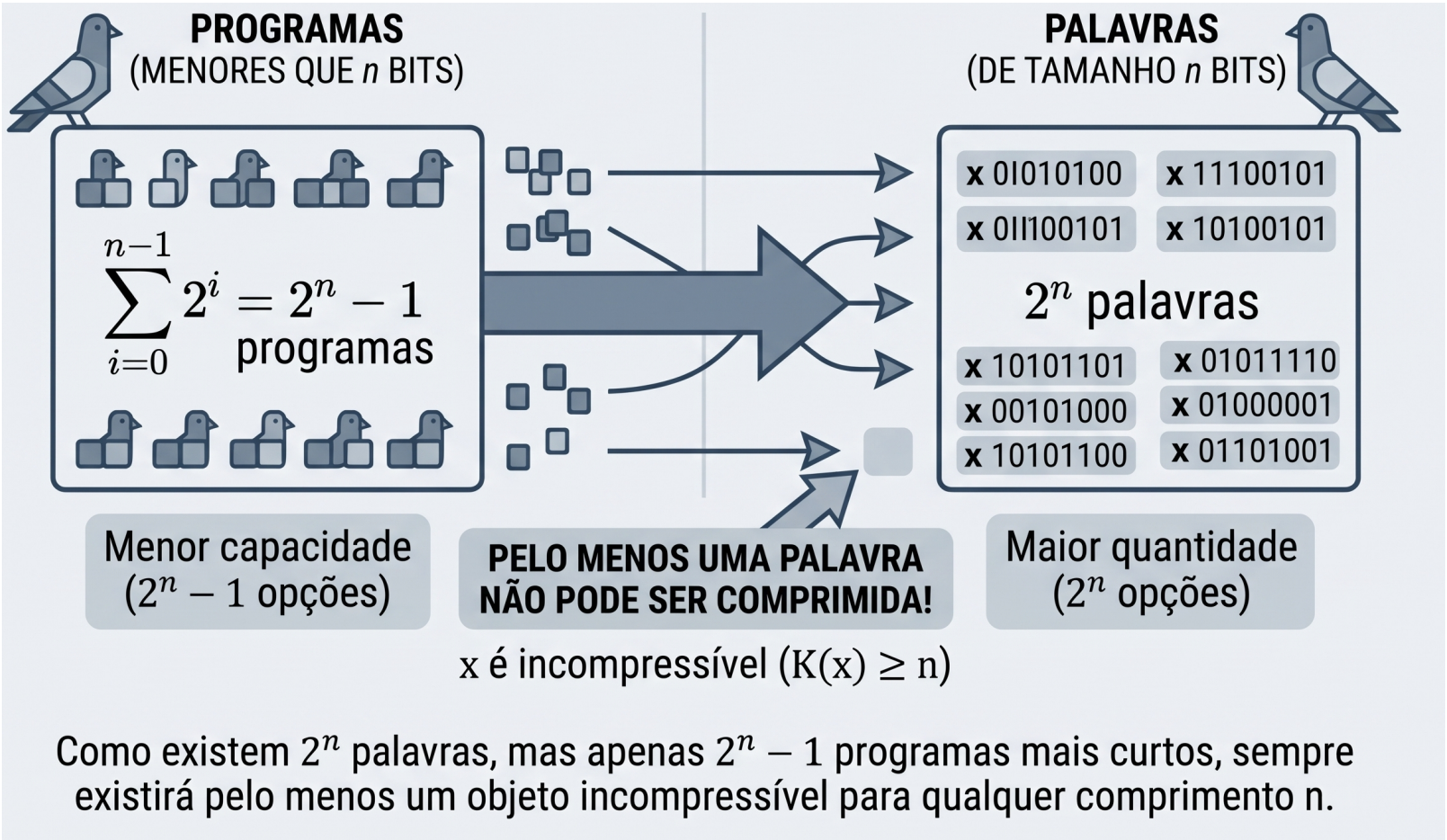


Figura 3: A violação do Princípio da Casa dos Pombos em relação à Complexidade de Kolmogorov.

A Violação do Princípio em Strings 2^n, sendo $n = 4$

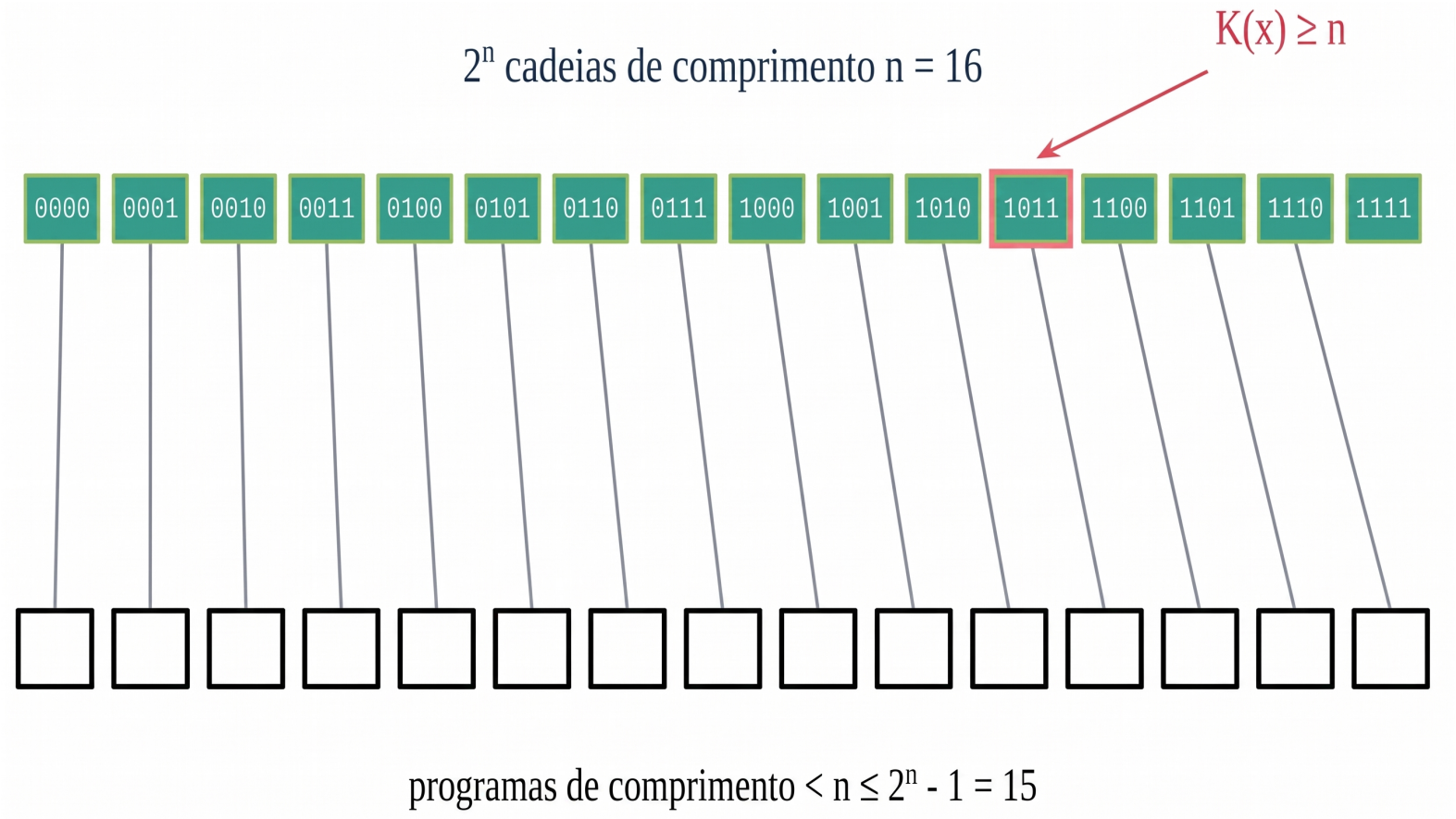


Figura 4: A violação do Princípio da Casa dos Pombos em relação à Complexidade de Kolmogorov para uma string de $n = 4$.

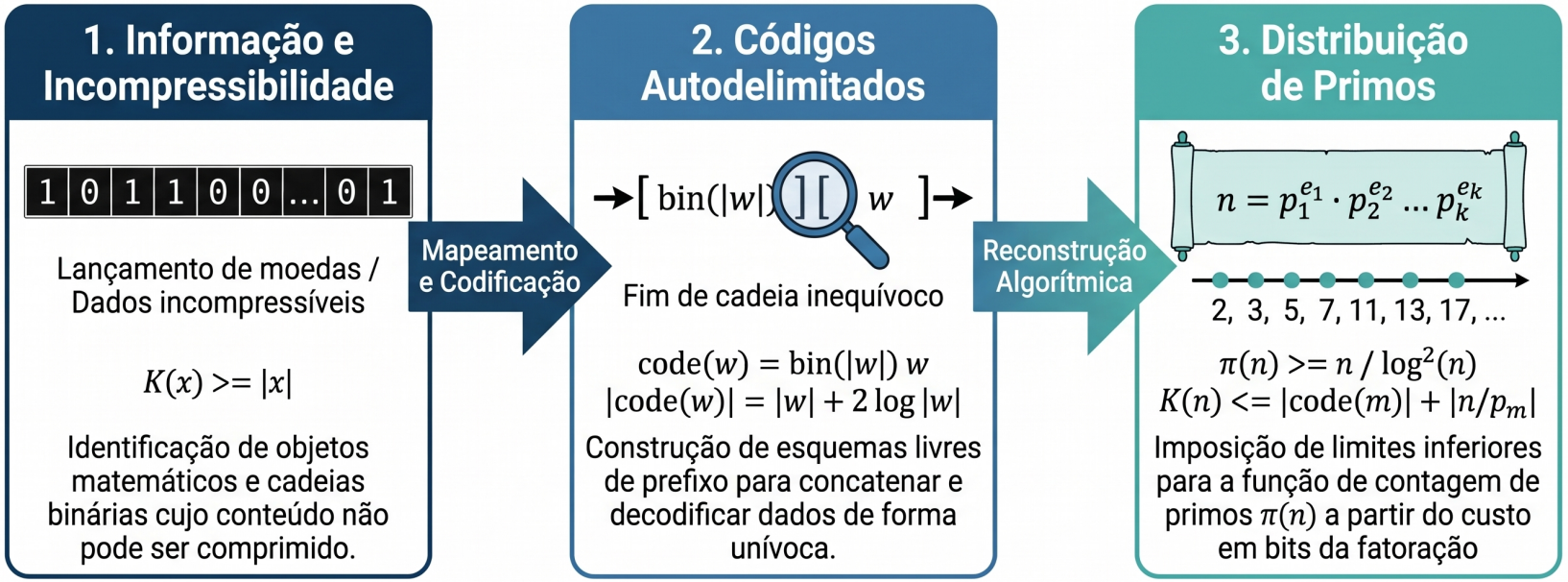
Sumário

1. Introdução
2. Problema de Pesquisa
3. Objetivos Geral e Específicos
4. Resultados e Entregáveis
5. Cronograma
6. Referências

Problema de Pesquisa: O Hiato nas Bibliotecas de Lean 4

- O ecossistema atual do Lean 4 divide-se entre a **Mathlib** (rica em Teoria dos Números) e a **Complexitylib** (que formaliza a complexidade descritiva $C_M(x)$ e a existência de strings binárias incompressíveis);
- Contudo, a **Complexitylib** opera puramente sobre strings isoladas (`List Bool`), carecendo da infraestrutura necessária para demonstrar limites inferiores em estruturas aritméticas:
  1. **Ausência de Códigos Autodelimitados:** Na complexidade simples, concatenar descrições exige esquemas livres de prefixo (*self-terminating codes*) para demarcar fronteiras de decodificação sem perda assintótica;
  2. **Ponte Informacional-Aritmética:** Acoplar a reconstrução algorítmica à fatoração única e à enumeração de primos da `Mathlib`;
  3. **Transferência Assintótica:** Formalizar como a redução no comprimento de códigos binários impõe limites inferiores sobre a função de contagem de primos $\pi(n)$.

Problema de Pesquisa



Contribuição Científica: Formalização de uma teoria construtiva de códigos livres de prefixo para derivar estimativas de densidade de primos sobre a *Complexitylib*.

Figura 5: Arquitetura da formalização: o diagrama ilustra o fluxo metodológico do projeto, detalhando como o princípio abstrato da incompressibilidade de Kolmogorov (1) é estendido por uma teoria de códigos autodelimitados (2) para estabelecer limites inferiores quantitativos na distribuição de números primos (3), evidenciando a extensão proposta sobre a base da `Complexitylib`.

Problema de Pesquisa

Problema Central de Pesquisa

Como estender a infraestrutura de Complexidade de Kolmogorov da `Complexitylib` em Lean 4 com esquemas formais de **codificação autodelimitada** para estabelecer, por pura incompressibilidade algorítmica, a infinitude dos números primos e limites inferiores assintóticos para a sua distribuição $\pi(n)$?

Sumário

1. Introdução
2. Problema de Pesquisa
3. Objetivos Geral e Específicos
4. Resultados e Entregáveis
5. Cronograma
6. Referências

Objetivo Geral

Incompressibilidade Aplicada à Distribuição de Primos

Desenvolver uma formalização mecanicamente verificada em Lean 4 que integre a **Complexitylib** e a **Mathlib** por meio de códigos autodelimitados, demonstrando como melhorias na eficiência descritiva de pares de números naturais se traduzem diretamente em limites inferiores progressivamente mais fortes para a função de contagem de primos $\pi(n)$.

Objetivos Específicos I

1. **Ponte Aritmética e Integração com a Complexitylib:**
   - Conectar o módulo de incompressibilidade de strings (`Cslib.Kolmogorov`) à representação binária canônica de números naturais;
   - Formalizar o princípio de reconstrução injetora como limite superior para a complexidade de Kolmogorov de inteiros;
   - Demonstrar a infinitude dos números primos via incompressibilidade da codificação da tupla de expoentes de fatoração.

2. **Formalização de Códigos Autodelimitados (*Self-Terminating Codes*):**
   - Construir funções de codificação livre de prefixo por intercalação de bits ($\overline{w}$) e por prefixação logarítmica de comprimento ($\text{code}(w) = \overline{\text{bin}(|w|)}w$);
   - Demonstrar formalmente a decodificação única em cadeias concatenadas e caracterizar suas cotas exatas de comprimento;
   - Generalizar o esquema para codificações autodelimitadas iteradas.

Objetivos Específicos II

3. **Derivação de Limites Inferiores para $\pi(n)$ (Teorema Fraco dos Primos):**
   - Formalizar a reconstrução algorítmica de um inteiro n a partir do índice do seu maior fator primo m e do quociente n/p_m;
   - Estabelecer a redução analítica que converte cotas superiores do m-ésimo primo (p_m) em limites inferiores para $\pi(n)$;
   - Demonstrar a cota assintótica $\pi(n) \geq \frac{n}{\log^2 n}$ e sua otimização via códigos iterados.

4. **Verificação Mecânica e Contribuição de Biblioteca:**
   - Garantir que todas as definições, lemas de comprimento e teoremas de limites inferiores sejam aceitos pelo verificador de tipos do Lean 4 (livres de `sorry`), estruturando um módulo extensível sobre a `Complexitylib`.

Sumário

1. Introdução
2. Problema de Pesquisa
3. Objetivos Geral e Específicos
4. Resultados e Entregáveis
5. Cronograma
6. Referências

Resultados Esperados: Hierarquia de Formalização

Abordagem	**Codificação / Custo**	**Limite de $\pi(n)$ Verificado**
1. Qualitativo	Expoentes da fatoração $n = p_1^{e_1} \cdots p_k^{e_k}$	Infinitude de primos $(\pi(n) \to \infty)$
2. Elementar	Intercalação de bits $\overline{w}$ Custo: $2\lvert w\rvert$	Cota polinomial $\pi(n) \geq \sqrt{n}$
3. Compacto	Prefixo de tamanho $\text{code}(w) = \overline{\text{bin}(\lvert w\rvert)}w$ Custo: $\lvert w\rvert + 2\log\lvert w\rvert$	**Teorema Fraco dos Primos:** $\pi(n) \geq \frac{n}{\log^2 n}$
4. Iterado	Prefixo recursivo Custo: $\log\lvert w\rvert + 2\log\log\lvert w\rvert$	Densidade otimizada $\pi(n) \geq \frac{n}{\log n(\log\log n)^2}$

Tabela 1: Impacto da compacidade de códigos no limite de $\pi(n)$.

Entregáveis

Dentro do escopo das atividades propostas em (SCHÖNING; PRUIM, 1998), estão previstos os seguintes entregáveis:

Entregáveis do Projeto

- Definições, lemas e teoremas das soluções dos exercícios, implementados e formalmente verificados em Lean 4 (livres de `sorry`);
- Código organizado, documentado e reproduzível a partir de um repositório público no GitHub;
- Relatório em PDF com o detalhamento passo a passo das soluções dos exercícios.

Sumário

1. Introdução
2. Problema de Pesquisa
3. Objetivos Geral e Específicos
4. Resultados e Entregáveis
5. Cronograma
6. Referências

Cronograma de Execução

Período	Meta de Formalização em Lean 4
02/11 – 08/11	**Integração de Base:** Configuração do ambiente, importação da `Complexitylib` e adaptação das definições de sequências.
09/11 – 15/11	**Códigos Autodelimitados:** Formalização das funções de codificação livres de prefixo ($\overline{w}$ e code(w)) e provas de decodificação única.
16/11 – 22/11	**Ponte de Incompressibilidade:** Conexão dos lemas de contagem com a aritmética e formalização da reconstrução bivariada (Ex. 9.2 e 9.3).
23/11 – 29/11	**Limites na Distribuição de Primos:** Demonstração da infinitude e do Teorema Fraco dos Primos ($\pi(n) \geq \frac{n}{\log^2 n}$) via incompressibilidade.
30/11 – 04/12	**Consolidação e Entrega:** Eliminação de eventuais `sorry`, elaboração do relatório técnico e submissão do repositório final.

Sumário

1. Introdução
2. Problema de Pesquisa
3. Objetivos Geral e Específicos
4. Resultados e Entregáveis
5. Cronograma
6. Referências

Referências I


SCHÖNING, U.; PRUIM, R. J. *Gems of Theoretical Computer Science*. Berlin, Heidelberg: Springer-Verlag, 1998. 77–83 p.